

DESCRIZIONE DEL PROGETTO

Black-Jack-OG è un progetto del gioco del Blackjack sviluppato in linguaggio C come progetto per il Laboratorio di Informatica ITPS UNIBA.

Il software implementa le regole base del Black Jack e offre un'interfaccia testuale per giocare, gestire gli utenti e salvare i risultati delle ultime mani tramite database SQLite.

MANUALE TECNICO

- **Requisiti funzionali**

- Il sistema deve consentire agli utenti di registrarsi e autenticarsi con username e password.
- Ogni utente può giocare più partite di Black Jack, gestendo il proprio saldo.
- Il sistema deve salvare le ultime 10 mani di ogni utente, comprensive di esito e puntata.
- Devono essere disponibili funzionalità per visualizzare le ultime 10 mani giocate.
- Deve esserci la possibilità di uscire e riprendere una partita.

- **Requisiti non funzionali**

- Il sistema deve funzionare in ambiente Windows e Linux.
- I dati sensibili (password) non devono essere visualizzati a schermo durante l'inserimento.
- Il sistema deve essere usabile tramite terminale.
- Il database deve essere persistente tra una sessione e l'altra.

- **Requisiti di sistema**

- Il software deve essere compilabile con GCC.
- È richiesto SQLite3 come dipendenza esterna.

ARCHITETTURA DEL SOFTWARE

Il progetto è strutturato in file C:

- main.c: punto di ingresso, gestione del flusso principale.
- user.c / user.h: gestione utenti, autenticazione, saldo, storico mani.
- deck.c / deck.h: gestione del mazzo di carte.
- blackjack.c / blackjack.h: logica del gioco Black Jack.
- database.c / database.h: interfacciamento con SQLite per la persistenza.
- utils.c / utils.h: funzioni di utilità e di supporto per input/output, validazione e gestione dell'interfaccia testuale.

DATABASE

Il progetto utilizza un database SQLite per la memorizzazione persistente dei dati relativi agli utenti e alle mani giocate.

Tabella: utenti: contiene tutte le informazioni necessarie per identificare e gestire ciascun giocatore.

Campi:

- id: identificativo numerico univoco dell'utente (chiave primaria, autoincrementale).

- nome: nome dell'utente.
- cognome: cognome dell'utente.
- username: nome utente scelto, univoco all'interno del sistema.
- password: password associata all'utente.
- saldo: saldo attuale dell'utente, aggiornato ad ogni mano giocata.

Tabella: mani giocate: dedicata alla memorizzazione delle mani di Black Jack giocate da ciascun utente.

Campi:

- id: identificativo numerico univoco della mano (chiave primaria, autoincrementale).
- user_id: identificativo dell'utente che ha giocato la mano.
- puntata: importo puntato dall'utente per quella mano.
- esito: esito della mano, che può essere "WIN" (vittoria), "LOSE" (sconfitta) o "PUSH" (pareggio).

ESEMPIO D'USO

1. L'utente si registra/accede.
2. L'utente visualizza il saldo e sceglie la puntata.
3. Il sistema gestisce la partita (distribuzione carte, domande, esiti).
4. Alla fine di ogni mano, il risultato viene memorizzato nel database;
5. L'utente può visualizzare in ogni momento lo storico delle ultime mani.

CASI D'USO

Registrazione:

1. L'utente seleziona "Registra nuovo utente".
2. Inserisce username e password.
3. Il sistema verifica la disponibilità e crea l'utente.

Gioco:

1. L'utente accede, sceglie la puntata.
2. Gioca la mano.
3. Visualizza l'esito.
4. Il sistema aggiorna il saldo e lo storico.

Visualizzazione storico:

1. L'utente seleziona "Visualizza ultime 10 mani".
2. Il sistema mostra puntata ed esito delle ultime 10 mani.

DESCRIZIONE FUNZIONI

db.h – Gestione del Database:

apriDatabase(sqlite3 db):**

Questa funzione apre la connessione al database SQLite. Riceve come parametro il puntatore al puntatore del database e si occupa di inizializzarlo. È fondamentale chiamarla prima di qualsiasi operazione che coinvolga il database.

creaTabellaUser(sqlite3* db):

Crea la tabella degli utenti nel database, se questa non esiste già. La funzione è essenziale

per strutturare il database e garantire che siano presenti le colonne necessarie per la memorizzazione delle informazioni degli utenti.

registraUtente(sqlite3* db, char* nome, char* cognome, char* username, char* password):

Permette di inserire nel database un nuovo utente, specificando nome, cognome, username e password. È la funzione chiamata durante la registrazione di un nuovo account.

loginDb(sqlite3* db, const char* username, const char* password):

Si occupa di verificare le credenziali di accesso di un utente. Restituisce 1 se il login è riuscito, 0 altrimenti.

caricaUtente(sqlite3* db, const char* username, Utente* utente):

Recupera dal database tutte le informazioni relative a un utente dato l'username, e le salva nella struttura Utente passata come parametro. Restituisce 1 se il caricamento ha successo.

utenteEsiste(sqlite3* db, const char* username):

Verifica se un determinato username è già presente nel database. Restituisce 1 se esiste, 0 altrimenti.

aggiornaSaldo(sqlite3* db, const char* username, int nuovoSaldo):

Aggiorna il saldo dell'utente specificato nel database, impostando il nuovo valore fornito come parametro.

salvaMazzoUtente(sqlite3* db, Carta* mazzo, const char* username):

Salva lo stato attuale del mazzo di carte associato a un utente. Utile per permettere la ripresa di una partita in un secondo momento.

caricaMazzoUtente(sqlite3* db, const char* username):

Permette di caricare dal database lo stato del mazzo di carte associato all'utente specificato.

creaTabellaMano(sqlite3* db):

Crea la tabella nel database per memorizzare le mani giocate dagli utenti. Questa tabella viene utilizzata per lo storico delle ultime 10 mani.

aggiungi_mano(sqlite3* db, int user_id, int puntata, const char* esito):

Inserisce una nuova mano giocata dall'utente nel database, registrando la puntata e l'esito della mano ("WIN", "LOSE" o "PUSH").

deck.h – Gestione del Mazzo di Carte

pilaToArray(Carta* top, int size):

Converte una pila di carte (struttura tipica di una pila LIFO) in un array di puntatori a Carta, facilitando così operazioni che richiedono l'accesso casuale alle carte.

contaCarte(Carta* top):

Conta il numero di carte presenti in una pila a partire dall'elemento di cima.

mescolaMazzo(Carta top):**

Mescola casualmente le carte nel mazzo, modificando l'ordine degli elementi nella pila.

creaMazzo(Carta top):**

Crea un nuovo mazzo standard di carte, inserendo tutte le carte nella pila a partire da un puntatore nullo alla cima.

push(Carta top, char* seme, char* valore, int punti):**

Inserisce una nuova carta in cima alla pila, specificando seme, valore e punteggio.

pop(Carta top):**

Estrae e restituisce la carta in cima alla pila, rimuovendola dal mazzo.

game.h – Logica del Gioco

calcolaPunteggio(Carta mano[], int numCarte):

Calcola il punteggio totale di una mano di carte, secondo le regole del Black Jack (inclusa la gestione degli assi come 1 o 11).

stampaCarteAffiancate(Carta carte[], int n):

Stampa su schermo n carte affiancate, facilitando la comprensione visiva della mano.

stampaCarteAffiancateConCoperta(Carta carte[], int n):

Stampa le carte affiancate ma con la prima carta coperta (utile per il banco).

turnoGiocatore(Utente* utente, Carta mazzo, Carta manoGiocatore[], Carta manoBanco[], int* numCarteGiocatore, int numCarteBanco, Carta* cartaEstratta, int* puntata):**

Gestisce le azioni del turno del giocatore: richiesta carta, stand, double down, aggiornamento del punteggio e della puntata.

turnoBanco(Utente* utente, Carta mazzo, Carta manoBanco[], Carta manoGiocatore[], int numCarteBanco, int numCarteGiocatore, Carta* cartaEstratta, int punteggioGiocatore):**

Gestisce il comportamento automatico del banco durante il proprio turno, secondo le regole del Blackjack.

partita(Utente* utente, int nuova):

Gestisce l'intero ciclo di gioco per una partita, dalla distribuzione delle carte all'aggiornamento dei risultati, passando per i turni di giocatore e banco.

user.h – Gestione Utenti

registraUtente():

Funzione interattiva che guida l'utente nella procedura di registrazione di un nuovo account.

login(Utente* utente):

Permette a un utente di autenticarsi inserendo le proprie credenziali. Se il login va a buon fine, la struttura Utente viene aggiornata.

signUp(Utente* utente):

Guida interattiva per la creazione di un nuovo utente, raccoglie i dati necessari e li passa alle funzioni di registrazione.

utils.h – Funzioni di Utilità e Interfaccia

printCentered(const char* message):

Stampa un messaggio centrato orizzontalmente nel terminale, migliorando la leggibilità dell'interfaccia testuale.

getPassword(char* password, int maxLength):

Permette l'inserimento sicuro di una password, nascondendo i caratteri digitati.

getScelta(int* menu):

Acquisisce la scelta dell'utente dal menù, gestendo eventuali errori di input.

printMenuLogin():

Mostra a schermo il menu di login.

printMenuScelta(char* username, int saldo):

Stampa a video il menu principale dopo il login, mostrando l'username e il saldo attuale.

printChiusuraProgramma():

Visualizza un messaggio di chiusura al termine del programma.

printOpzioneNonValida():

Avvisa l'utente se viene selezionata un'opzione non valida nel menu.

printUsernameSaldo(Utente utente):

Stampa a schermo l'username e il saldo attuale dell'utente.

printMescolamento():

Mostra un menu relativo alle operazioni di mescolamento del mazzo.

manoIniziale(Utente utente, Carta mazzo, Carta** cartaEstratta, Carta manoBanco[], Carta manoGiocatore[]):**

Gestisce la distribuzione delle carte all'inizio di una mano sia per il banco che per il giocatore.

printCarteBancoGiocatore(Utente* utente, int punteggioBanco, int punteggioGiocatore, int puntata, Carta manoBanco[], Carta manoGiocatore[], int offsetBanco, int offsetGiocatore, int finale):

Stampa lo stato attuale delle carte di banco e giocatore durante la partita, con opzioni di visualizzazione avanzate.

printCarteBancoGiocatoreCoperta(Utente* utente, Carta manoBanco[], Carta manoGiocatore[], int offsetBanco, int offsetGiocatore):

Mostra le carte del banco (con una coperta) e del giocatore, utile per la fase iniziale.

getPuntata(Utente* utente):

Permette all'utente di inserire l'importo da puntare nella mano corrente, controllando i limiti rispetto al saldo.

risultato(Utente* utente, int punteggioBanco, int punteggioGiocatore, int puntata, int numCarteGiocatore):

Calcola e visualizza l'esito della mano, aggiornando il saldo dell'utente e lo storico nel database.

printMenuGiocatore(int numCarteGiocatore):

Stampa il menu delle opzioni disponibili al giocatore durante il proprio turno.

contaCarteNelMazzo(Carta* mazzo):

Restituisce il numero di carte rimanenti nel mazzo.

serializzaMazzo(Carta* mazzo):

Converte il mazzo in una stringa serializzata, utile per il salvataggio su database.

deserializzaMazzo(const char* str):

Ricostruisce il mazzo di carte da una stringa serializzata.

liberaMazzo(Carta* mazzo):

Libera la memoria occupata dal mazzo di carte.

stampaUltime10Mani(sqlite3* db, Utente utente):

Recupera e mostra a schermo l'elenco delle ultime 10 mani giocate dall'utente, con dettagli su puntata ed esito.

AUTORI

Luca Perrone & Onofrio Soranno